AURA

System Design and Architecture

Autonomous University Response and Action

Department of Computer Science & Engineering
TKM College of Engineering

This document presents the system design and architecture of AURA, a Java-based campus issue and suggestion management platform developed to improve transparency, anonymity, prioritization, and accountability in academic institutions.

1. Project Context and Design Goal

AURA addresses the problem of fragmented campus issue reporting, where students often raise concerns through informal and unstructured channels. This makes it difficult for administrators to track, prioritize, and resolve problems efficiently. The platform introduces a structured digital workflow that combines anonymous reporting, status tracking, student ownership, and administrative accountability.

2. Architectural Vision

The architecture of AURA follows a disciplined four-layer model: user interface, service logic, data access, and database storage. This separation improves maintainability, reduces coupling, and ensures that business rules remain independent from persistence logic and interface design.

3. Four-Layer Architecture

Layer	Responsibility	Key Components
Presentation Layer	Displays screens and captures user actions	LoginFrame, StudentDashboardFrame, AdminDashboardFrame, S
Service Layer	Contains business logic and validation	AuthService, SubmissionService, DatabaseSubmissionService, H
Data Access Layer	Handles all JDBC operations and database interaction	UserDAO, SubmissionDAO, StudentReceiptDAO, SubmissionHyp
Database Layer	Stores project data in a relational model	Supabase PostgreSQL tables such as users, submissions, receipt

The key rule is the downward dependency flow: UI → Service → DAO → Database. The graphical layer does not contain SQL logic or direct database calls. This separation is essential for architecture clarity and security.

4. Core Design Principles

- Separation of concerns: each class has a clearly defined responsibility.
- Security by design: authentication, password hashing, validation, and access control are enforced in the application logic.
- Privacy by architecture: public submission data is separated from private ownership data.
- Auditability: all issue status transitions are recorded through history records.
- Maintainability: modular code structure supports future enhancement and testing.

5. Student Module Design

The student module allows a user to register, log in, submit issues, vote on issues that matter to them, and track their own reports. The system supports both issue reporting and suggestions, enabling constructive feedback beyond complaint handling.

The student can provide: title, description, category, location, priority, and optional supporting media. Each submission is stored as an anonymous public record so that student identity is not exposed to the administrative queue.

6. Admin Module Design

The administrative module provides a structured issue queue. Administrators can filter and review all submissions, inspect details, update the status lifecycle, assign tasks, and add resolution notes. This ensures that campus problems are handled in a transparent and accountable manner.

The admin flow is designed to prioritize action rather than identity disclosure. Administrators can manage the system without seeing the author of each public ticket, reinforcing the project's privacy-first approach.

7. Hype and Prioritization Design

AURA includes a hype engine that lets students support submissions they also consider important. This creates a crowd-prioritized list of issues and helps urgent problems rise to the top. Duplicate voting is prevented through unique constraints and logic in HypeService and SubmissionHypeDAO.

8. Status Lifecycle and Workflow

The system follows a five-state process: PENDING → ASSIGNED → IN_PROGRESS → RESOLVED / REJECTED. This lifecycle is crucial because it transforms the reporting platform from a passive form into a true issue management system with tracked progress and closure evidence.

Step	Meaning
PENDING	The issue is submitted and awaits administrative review
ASSIGNED	A responsible authority has been assigned to the issue
IN_PROGRESS	The issue is actively being addressed
RESOLVED	The issue has been addressed and closed
REJECTED	The issue is not valid or not accepted under the defined policy

9. Database Design and Data Model

The data model is designed to support secure, structured, and query-friendly operation. The main tables are users, submissions, student_submission_receipts, submission_hype, resolution_notes, and submission_history.

The most important design decision is that the submissions table does not store student identity. Instead, the student receipt mapping is stored separately and used only when a student needs to view their personal submissions. This is the core privacy mechanism of the system.

Table	Purpose
users	Stores login and role information for students and administrators
submissions	Stores anonymous public issue and suggestion records
student_submission_receipts	Links students to their own submissions without exposing identity to admin flows
submission_hype	Stores peer upvotes and prevents double voting
resolution_notes	Stores official admin action records linked to a ticket

submission_history	Tracks every state transition for auditability

10. Security Architecture

AURA incorporates multiple layers of security to protect confidential data and maintain integrity. Email validation restricts access to official institutional addresses, password hashing uses BCrypt, and SQL is accessed via prepared statements to prevent injection attacks.

The system also enforces role-based authorization, meaning that students cannot access admin functions and admins cannot query the private receipt data used by students. This is enforced in both UI and service logic and is a strong architectural safeguard.

11. Technical Stack

Component	Choice
Language	Java 17
UI	Java Swing with FlatLaf
Persistence	JDBC with PostgreSQL
Database	Supabase PostgreSQL
Security	BCrypt, domain validation, prepared statements
Build Tool	Apache Maven
Testing	JUnit 5

12. Architectural Strengths

AURA demonstrates a well-structured and practically valid software architecture. It maintains proper boundary separation, active privacy protection, clear role division, efficient persistence design, and auditable state transitions. These qualities reflect strong academic and professional engineering practice.

13. Conclusion

The system design and architecture of AURA are built around a real campus need: transparent, secure, and prioritized issue reporting. The project combines strong backend logic, institutional security rules, and a user-friendly desktop experience into one coherent design. This makes AURA a well-balanced academic project and a realistic software solution for campus governance.